

Relational Database Management System (SQL) :

1. Foundational Database Theory and Structure

- **PRIMARY KEY** : A unique identifier for every row that enforces **Entity Integrity** . It ensures that no two records are identical and providing a reliable means of row-level access.
- **VARCHAR** : A variable-character data type used to store string-based information such as names or cities.
- **INT** : A data type reserved for whole numbers, facilitating mathematical operations and efficient indexing.

2. Database Initialization: Creation and Population

1. **Data Definition Language (DDL)** : The first step involves creating the structural blueprint. We use the CREATE TABLE command to declare the table name, column names, and their respective constraints.
2. **Data Manipulation Language (DML)** : Once the structure exists, the INSERT INTO command is utilized to populate the blueprint with state-specific data.

SQL Implementation: students Table



```
1  -- Create secondary table 'city_info'
2  • CREATE TABLE city_info (
3      city VARCHAR(50),
4      state VARCHAR(50),
5      pin_code INT
6  );
7
8  -- Insert data into 'city_info'
9  • INSERT INTO city_info (city, state, pin_code) VALUES
10 ('Mumbai', 'Maharashtra', 400067),
11 ('Sangli', 'Maharashtra', 416406),
12 ('Thiruvananthapuram', 'Kerala', 695001),
13 ('Pune', 'Maharashtra', 400000),
14 ('Ahmedabad', 'Gujarat', 306780);
```

3. Data Retrieval, Filtering, and Sorting

- **SELECT** : Dictates the projection of specific attributes.
- **WHERE** : Filters the result set based on logical and mathematical predicates.
- **ORDER BY** : Determines the sort order. While ASC is the default, DESC is used for high-to-low prioritization.

The screenshot shows a SQL query editor with a toolbar at the top. The query is as follows:

```
1 -- Filtering for specific performance and geography
2 • SELECT name, marks, city
3 FROM students
4 WHERE marks <= 85 AND (city = 'Mumbai' OR city = 'Thiruvananthapuram')
5 ORDER BY marks DESC;
6
```

Below the query editor is a 'Result Grid' section. It includes a 'Filter Rows' input field, an 'Export' button, and a 'Wrap Cell Content' checkbox. The results are displayed in a table with three columns: name, marks, and city.

	name	marks	city
▶	Shriya	85	Thiruvananthapuram
	Atharva	70	Mumbai
	Lakshmi	45	Mumbai

4. Advanced Querying: Nested Subqueries

Subqueries allow for dynamic filtering by nesting one query within another. The database engine follows a specific order of evaluation: the **inner query** is executed first to generate a temporary result set, which is then passed to the **outer query**. The **IN operator** is the standard method for comparing values against the list generated by the subquery.

```
1 • SELECT name, city FROM students
2 WHERE city IN (
3     SELECT city FROM students WHERE marks > 80
4 )
```

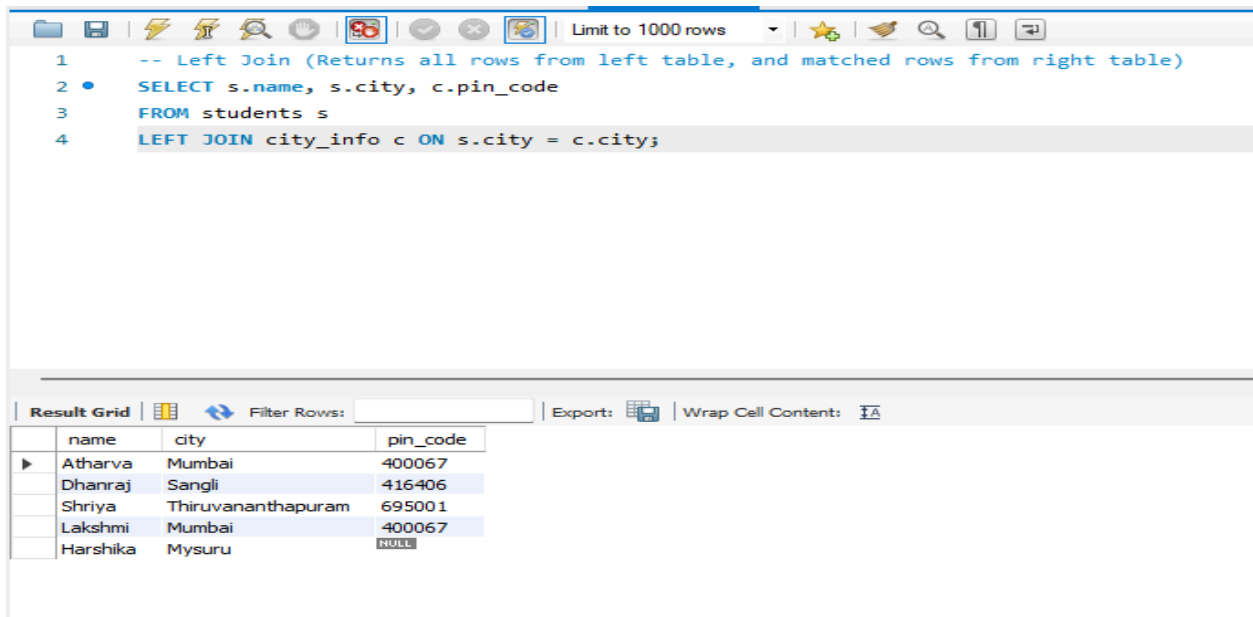
Result Grid | Filter Rows: | Export: | Wrap Cell Content: |

	name	city
▶	Dhanraj	Sangli
	Shriya	Thiruvananthapuram
	Harshika	Mysuru

5. Relational Logic: Table Joins

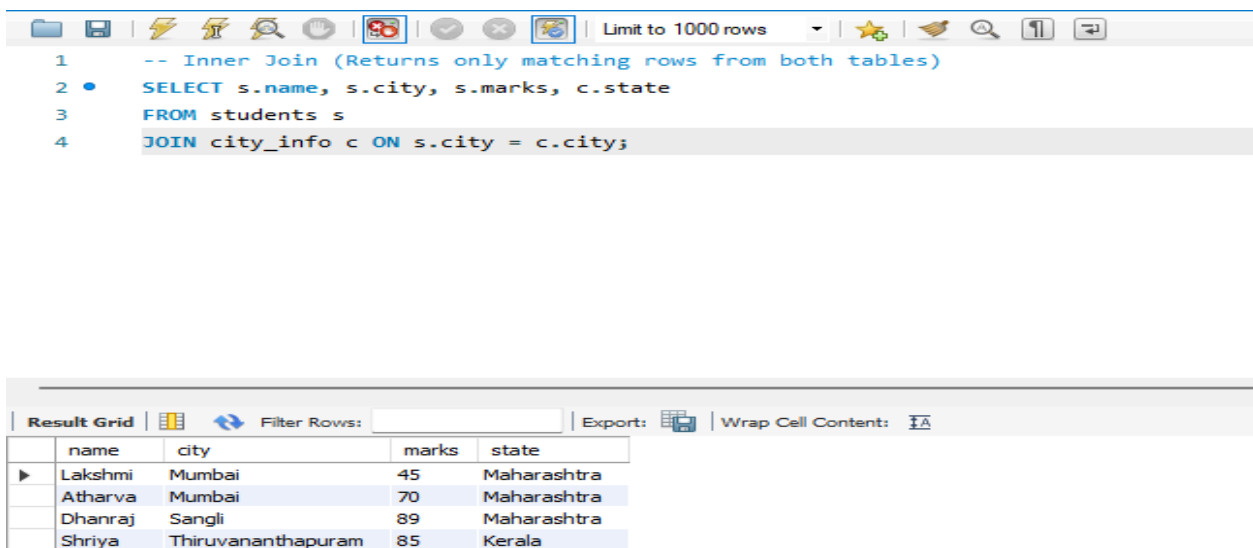
- **INNER JOIN** : A strict intersection; it only returns rows where a match exists in both datasets.
- **LEFT JOIN** : An inclusive operation that prioritizes the "Left" table (the one declared first in the FROM clause). It returns all records from the left table and fills in matching data from the right; if no match exists, it returns a NULL.

LEFT JOIN Execution



The screenshot shows a SQL IDE with a query editor and a result grid. The query is a LEFT JOIN between 'students' and 'city_info' on the 'city' column. The result grid displays the following data:

	name	city	pin_code
▶	Atharva	Mumbai	400067
	Dhanraj	Sangli	416406
	Shriya	Thiruvananthapuram	695001
	Lakshmi	Mumbai	400067
	Harshika	Mysuru	NULL



The screenshot shows a SQL IDE with a query editor and a result grid. The query is an INNER JOIN between 'students' and 'city_info' on the 'city' column. The result grid displays the following data:

	name	city	marks	state
▶	Lakshmi	Mumbai	45	Maharashtra
	Atharva	Mumbai	70	Maharashtra
	Dhanraj	Sangli	89	Maharashtra
	Shriya	Thiruvananthapuram	85	Kerala

6. Programmatic Logic: The CASE Statement

The CASE statement implements conditional branching logic directly within a query. It is evaluated **top-to-bottom** ; once a condition is satisfied, the engine assigns the value and moves to the next row. This sequential evaluation is why thresholds must be ordered from

highest to lowest (e.g., check before 75). **Grading Logic Thresholds:**

```
1 • SELECT name, marks,  
2 CASE  
3 WHEN marks >= 90 THEN 'A+'  
4 WHEN marks >= 75 THEN 'A'  
5 WHEN marks >= 60 THEN 'B'  
6 WHEN marks >= 35 THEN 'C'  
7 ELSE 'Fail'  
8 END AS grade  
9 FROM students;
```

Result Grid			
Filter Rows:			
Export: Excel CSV PDF			
Wrap Cell Content: On Off			
	name	marks	grade
▶	Atharva	70	B
	Dhanraj	89	A
	Shriya	85	A
	Lakshmi	45	C
	Harshika	99	A+

7. Analytical Window Functions

- **DENSE_RANK()** : Assigns a sequential rank based on a value. Ties receive the same rank, and the subsequent rank is the next consecutive integer (preventing gaps in the ranking sequence).
- **PARTITION BY** : Functions as a logical "reset switch." It divides the result set into subsets (e.g., by city), allowing the window function to start its calculation over for each group.
- **FIRST_VALUE()** : Efficiently identifies the top-performing value (such as the highest scorer) within a partition without needing a complex subquery.

```

1  -- Partitioning by City to find running statistics or group evaluations
2  • SELECT name, city, marks,
3      SUM(marks) OVER (ORDER BY marks DESC) AS running_sum,
4      AVG(marks) OVER (PARTITION BY city) AS avg_city_marks
5  FROM students;

```

Result Grid		Filter Rows:		Export:		Wrap Cell Content:	
	name	city	marks	running_sum	avg_city_marks		
▶	Atharva	Mumbai	70	343	57.5000		
	Lakshmi	Mumbai	45	388	57.5000		
	Harshika	Mysuru	99	99	99.0000		
	Dhanraj	Sangli	89	188	89.0000		
	Shriya	Thiruvananthapuram	85	273	85.0000		